



**Simulación Monte-Carlo de epidemias paralela; Proyecto Final**

Elida Asneisy Payano Rodríguez

Instituto Tecnológico de Las Américas (ITLA)

Programación Paralela

Wilmer José Fariña

13 de agosto de 2026

# Informe Técnico de Rendimiento

Simulación Espacial Epidemiológica (Modelo SIR)

## I. Resumen Ejecutivo

El presente documento detalla la implementación y análisis arquitectónico de una simulación bidimensional de propagación de enfermedades basada en el modelo matemático SIR (Susceptibles, Infectados, Recuperados). El sistema fue desarrollado en C# y evalúa el impacto en el rendimiento de aplicar computación paralela frente a una aproximación estrictamente secuencial. Las simulaciones se ejecutaron sobre una grilla espacial de 1,000 x 1,000 individuos durante un periodo equivalente a un año calendario (365 días).

## 2. Arquitectura y Metodología de Paralelización

Para mitigar los cuellos de botella presentes en la versión secuencial, se rediseñó el bucle principal de actualización utilizando la librería *Task Parallel Library (TPL)*. El mapa bidimensional fue segmentado dividiendo las filas (*i*) mediante *Parallel.For*. Se resolvieron los riesgos inherentes a las condiciones de carrera (*Race Conditions*) a través de las siguientes implementaciones:

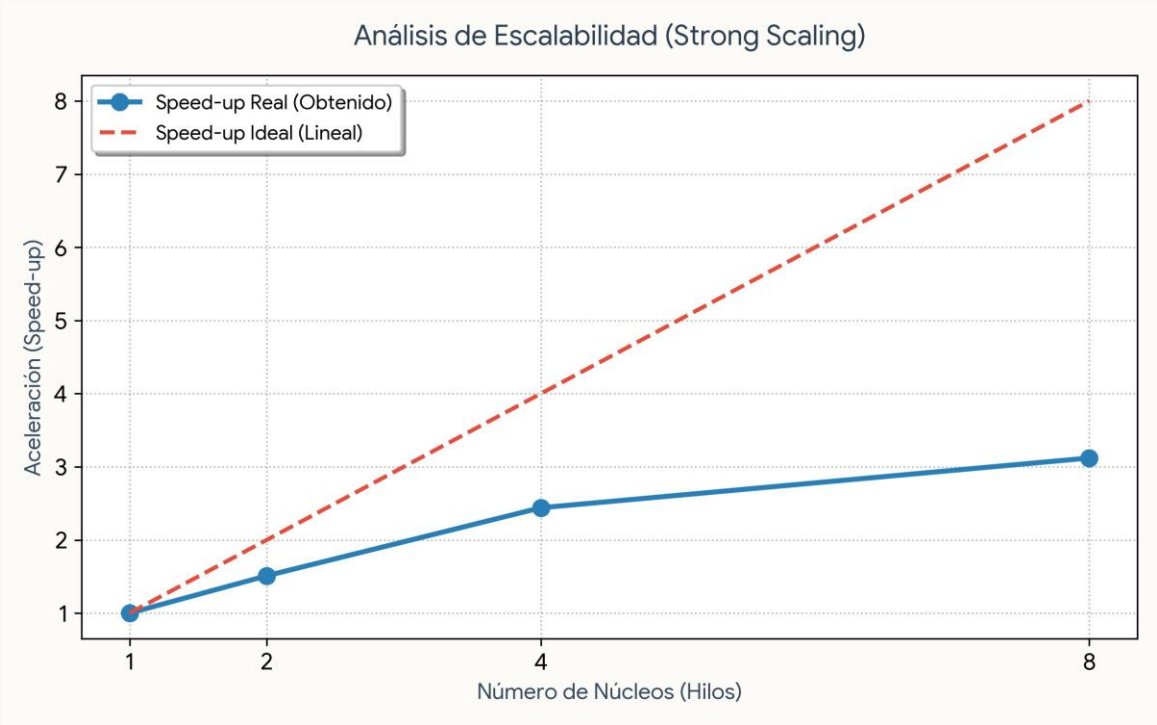
- Reducción Paralela:** Asignación de contadores de estado locales dentro del ámbito de cada hilo para prevenir bloqueos en memoria.
- Operaciones Atómicas:** Uso de *Interlocked.Add* para sincronizar los totales de manera segura sin necesidad de estructuras *lock* excesivas.

- **Aleatoriedad Segura:** Migración del generador clásico a *Random.Shared*, previniendo el colapso del objeto ante accesos concurrentes.

### 3. Experimentos de Strong Scaling (Speed-up)

El banco de pruebas se configuró para limitar el grado máximo de paralelismo a 1, 2, 4 y 8 núcleos físicos/lógicos. El *Speed-up* global de la aplicación (*S*) fue calculado bajo la relación  $S = T_1 / T_N$ , donde *T<sub>1</sub>* es el tiempo base. Los resultados empíricos obtenidos son:

| NÚCLEOS DISPONIBLES | TIEMPO TOTAL (MS) | ACELERACIÓN (SPEED-UP) |
|---------------------|-------------------|------------------------|
| 1 núcleo (Base)     | 7099              | 1.00x                  |
| 2 núcleos           | 4700              | 1.51x                  |
| 4 núcleos           | 2899              | 2.44x                  |
| 8 núcleos           | 2269              | 3.12x                  |



#### ***4. Análisis de Rendimiento (Ley de Amdahl)***

La paralelización demostró una reducción drástica del 68% en el tiempo de procesamiento total. No obstante, como se evidencia en la gráfica superior, el rendimiento divergió de la línea de escalado perfecto a partir del cuarto hilo de ejecución.

**Conclusión Técnica:** Esta estabilización térmica obedece a la Ley de Amdahl. El algoritmo propuesto es altamente dependiente del ancho de banda de la memoria (Memory-Bound) debido a la inmensa cantidad de operaciones de lectura/escritura (I/O) en una matriz de un millón de elementos. Adicionalmente, el trasvase de punteros de memoria (doble búfer) al final de cada iteración impone un candado estrictamente secuencial que impide que la aceleración tienda al infinito.

#### **5. Entregables Multimedia**

Para asegurar la consistencia del algoritmo y validar visualmente los parámetros estocásticos del modelo SIR, el sistema extrae representaciones gráficas del mapa en formato PNG (1 frame por día simulado). Posteriormente, estas secuencias de imágenes fueron compiladas vía FFmpeg, entregando un render de validación side-by-side (algoritmo secuencial vs paralelo), probando su idéntico comportamiento probabilístico bajo distintas arquitecturas de CPU.